PSTAT 132 - Homework 2

Aldo Nunez

2026-10-14

Question 1

Relational algebra:

$$\pi_{\text{CourseID}}\left(\sigma_{\text{StudentID}='1001'}(\text{ENROLLMENT})\right) - \pi_{\text{CourseID}}\left(\sigma_{\text{StudentID}='1002'}(\text{ENROLLMENT})\right)$$

SQL:

```sql
SELECT CourseID FROM ENROLLMENT WHERE StudentID = '1001'
EXCEPT
SELECT CourseID FROM ENROLLMENT WHERE StudentID = '1002';
```

Question 2

Relational algebra:

$$\pi_{\text{Name, Title}}\left(\sigma_{\text{Grade}='A'}\left(\text{STUDENT} \bowtie \text{ENROLLMENT} \bowtie \text{COURSE}\right)\right)$$

SQL:

```sql
SELECT S.Name, C.Title
FROM STUDENT S
JOIN ENROLLMENT E ON S.StudentID = E.StudentID
JOIN COURSE C ON E.CourseID = C.CourseID
WHERE E.Grade = 'A';
```

Question 3

Relational algebra:

$$\pi_{\text{Name, Title}}\left(\text{STUDENT} \bowtie \text{ENROLLMENT} \bowtie \text{COURSE}\right)$$

SQL:

```sql
SELECT S.Name, C.Title
FROM STUDENT S
JOIN ENROLLMENT E ON S.StudentID = E.StudentID
JOIN COURSE C ON E.CourseID = C.CourseID;
```

Question 4

Extended relational algebra:

$$\pi_{\text{ProductName, Price, Price}\times(1-\text{DiscountRate})\rightarrow\text{DiscountedPrice}}(\text{PRODUCT})$$

Question 5

```sql
WITH customer_totals AS (
    SELECT
        customer_id,
        SUM(order_amount) AS total_spend
    FROM orders
    WHERE order_date >= '2023-01-01'
      AND order_date <  '2023-02-01'
    GROUP BY customer_id
),
avg_spend AS (
    SELECT AVG(total_spend) AS avg_total_spend
    FROM customer_totals
)
SELECT
    ct.customer_id,
    ct.total_spend,
    a.avg_total_spend,
    ct.total_spend - a.avg_total_spend AS diff_from_avg,
    CASE
        WHEN ct.total_spend > a.avg_total_spend THEN 'Above average'
        WHEN ct.total_spend < a.avg_total_spend THEN 'Below average'
        ELSE 'Equal to average'
    END AS comparison
FROM customer_totals ct
CROSS JOIN avg_spend a
ORDER BY ct.total_spend DESC;
```

Question 6

I would implement this as a view:

```sql
CREATE VIEW customer_monthly_spend AS
SELECT
    customer_id,
    strftime('%Y-%m', order_date) AS order_month,
    SUM(order_amount) AS total_spend
FROM orders
GROUP BY customer_id, strftime('%Y-%m', order_date);
```

A view is the better fit because the same aggregation is reused across ten queries. It is defined once, stored in the database schema, and every query can reference it, whereas a CTE only exists within the single query it is written in and would have to be copied into all ten.

Question 7

1. Two intermediate relations are needed: (a) each customer's total spend for January 2023 (filter by date, `GROUP BY customer_id`, `SUM(order_amount)`), and (b) the average of those totals.
2. The CTE names the per-customer January totals once at the top of the query. The main query can then read from it twice: once to list customers and once to compute the average to compare against. Each logical step is separate and readable.
3. Repeating the aggregation in two nested subqueries duplicates logic. That is harder to read, and if one copy is edited (e.g., the date range), the two copies can silently drift apart. The CTE is a single source of truth.
4. No. A CTE is temporary and exists only while that one query executes. Nothing is saved, which is

why `SELECT * FROM january_completed_orders` fails in Demo 3.

5. No. A view adds a permanent schema object, which pays off when logic is reused across many queries or sessions. For a one-off or occasional analysis, a CTE is simpler and leaves nothing behind to maintain.

Question 8

```sql
WITH student_avg AS (
    SELECT
        StudentID,
        AVG(Grade) AS average_grade
    FROM ENROLLMENT
    GROUP BY StudentID
    HAVING AVG(Grade) >= 85
)
SELECT
    s.StudentID,
    s.Name,
    s.Major,
    sa.average_grade
FROM student_avg sa
JOIN STUDENT s
  ON s.StudentID = sa.StudentID
ORDER BY sa.average_grade DESC;
```

The CTE separates the aggregation and filtering step (average grade, keep only those at or above 85) from the join to STUDENT. The query reads in the same order as the problem's steps, and the join only touches the already-reduced set of successful students instead of mixing `GROUP BY`/`HAVING` with the join logic.